MULTICORE, part 2: parallel hybrid authentication in the real TLS handshake (end-to-end)

Date: 2026-11-11. Board: VisionFive 2 (JH7110, 4x SiFive U74), OpenSSL 3.5.9 (plain "opt" build, the same one the baseline used for hybrid). CPU at the **default setting** (`ondemand`, min 312.5 MHz), the same as the **BASELINE**. Follows the feasibility test in `multicore_feasibility_REPORT.md` (part 1).

What was built

Part 1 showed that ECDSA and ML-DSA verification run 1.5–1.65x faster on two cores. In the v1 design they could never overlap, because the ML-DSA signature only arrived after the handshake (in the HTTP response) while OpenSSL verified ECDSA during the handshake. **v2 moves the ML-DSA signature into the TLS 1.3 handshake itself.**

	v1 (all earlier results)	v2 (this report)
Where the ML-DSA-44 signature travels	HTTP response, after the handshake	**TLS 1.3 Certificate message**, as a certificate extension (private-use number `0xFE20`)
What it signs	`"hybrid-auth-v1"` ‖ TLS exporter (32 bytes)	`"hybrid-auth-v2"` ‖ SHA-256(ClientHello ‖ ServerHello)
When the client can verify it	after the handshake	**as soon as the Certificate arrives**, while OpenSSL still has to verify ECDSA
Client asks for it by	`PQ-Auth` HTTP header	an empty `0xFE20` extension in the ClientHello

Why the v2 signature is bound to the connection: ClientHello and ServerHello contain both random values and both key shares (X25519 + ML-KEM-768). A signature copied from another handshake would only verify with that handshake's ServerHello, but then the attacker doesn't have the matching private key share and can't finish the TLS handshake.

How the client uses two cores (`par` mode): 1. OpenSSL calls our extension callback when the Certificate message is parsed. 2. The callback hashes the recorded ClientHello and ServerHello and **wakes a persistent worker thread** that verifies ML-DSA-44 on another core. 3. OpenSSL continues on the main core and verifies the ECDSA CertificateVerify signature, **at the same time**. 4. After `SSL_connect` the client waits for the worker. The connection is used only if the certificate, ECDSA **and** ML-DSA all verified. Otherwise it's closed before any application data is sent.

`v2_seq` is the same design but verifies ML-DSA on the main core inside the callback, which isolates the design change from the effect of the second core.

Security tests (all passed)

Test	Result
Wrong ML-DSA-44 public key, `v2_seq`	handshake aborted by the client with a TLS `decrypt_error` alert
Wrong ML-DSA-44 public key, `v2_par`	connection rejected after the handshake, before any data is sent
Server that doesn't send the v2 signature (old server)	rejected ("signature missing"): no silent downgrade
Wrong pinned certificate	handshake fails (unchanged from v1)

Method

- Own server on the Mac (`src/hybrid_server.c`, now also sends the v2 extension), client on the board (`src/hybrid_client.c`, options `certext` and `par`).

- 4 modes, **1000 connections each**: 20 blocks x 50 runs, mode order rotating every block, so all modes see the same network conditions. 4000 connections, **0 failures**, ML-DSA verification confirmed in every hybrid block.
- Temperature 44.9 → 46.1 °C over the run.
- Files: `multicore_e2e_raw.csv` (`mode,block,run,tcp_ms,handshake_ms,e2e_ms,pq_verify_ms,wait_ms`), `multicore_e2e_stats.txt`, `multicore_e2e.log`, `scripts/bench_multicore_e2e.sh`.

Results: end-to-end time per connection (own server, LAN, default clock)

Mode	Best	p5	Median	Mean	p95	p99	Worst
Legacy (X25519 + RSA-3072)	15.37	18.08	**21.68**	21.79	24.62	27.29	35.33 ms
Hybrid v1 (one core)	20.72	22.78	**28.19**	28.27	31.93	34.03	40.40 ms
Hybrid v2, one core	21.07	22.72	**28.68**	28.60	32.19	33.72	40.87 ms
Hybrid v2, two cores	17.45	19.54	**24.57**	24.67	28.28	31.75	38.11 ms

Hybrid = X25519MLKEM768 + ECDSA P-256 + ML-DSA-44 in all three hybrid modes.

Paired per-block comparison

Comparison	Median difference	Per block	Faster in
v2 two cores vs v1	**–3.62 ms (–12.8 %)**	–3.55 ms (range –7.47 to –0.10)	**20/20 blocks**
v2 two cores vs v2 one core (effect of the 2nd core alone)	**–4.11 ms (–14.3 %)**	–4.04 ms (range –8.63 to –0.00)	**20/20 blocks**
v2 one core vs v1 (effect of the design change alone)	+0.49 ms (+1.7 %)	+0.43 ms	7/20 blocks: no real difference
v1 vs legacy	+6.51 ms (+30.0 %)	+6.20 ms	0/20
v2 two cores vs legacy	**+2.89 ms (+13.3 %)**	+2.78 ms (range –0.47 to +3.74)	2/20

Where the time goes (medians)

Mode	TCP	TLS handshake	ML-DSA verify (its own duration)	Waiting for ML-DSA after TLS
Legacy	1.38 ms	18.98 ms	none	none
v1	1.37 ms	19.30 ms	4.24 ms (after the handshake)	none
v2 one core	1.37 ms	25.95 ms (ML-DSA now inside)	3.99 ms	0.002 ms
v2 two cores	1.37 ms	21.87 ms	4.00 ms (overlapped with ECDSA)	**0.003 ms**

The ML-DSA verification still takes ~4 ms at the default clock, but on the second core it finishes before OpenSSL has finished ECDSA: the client waits essentially **0 ms** for it.

Comparison with the BASELINE (own server, LAN, default clock)

	Median	vs baseline legacy (21.61 ms)
Baseline legacy (earlier run)	21.61 ms	
Baseline hybrid v1 (earlier run)	27.95 ms	+6.34 ms
Legacy (this run)	21.68 ms	+0.07 ms (reproduces the baseline)
Hybrid v2, two cores	**24.57 ms**	**+2.97 ms**

Findings

1. **Multicore works end-to-end.** Moving the ML-DSA signature into the handshake and verifying it on a second core makes the full hybrid connection **3.6 ms faster (–12.8 %), in 20 of 20 blocks**.
2. **The second core is what delivers the gain.** The design change on its own is neutral (+0.5 ms, within noise); the second core is worth –4.1 ms.
3. **It matches the feasibility prediction:** part 1 measured –3.8 ms for the authentication pair at the default clock; end-to-end gives –3.6 to –4.1 ms.
4. **It more than halves the hybrid penalty:** v1 is +6.5 ms over legacy, v2 on two cores only +2.9 ms, at the default power-saving clock, with no extra heat (45–46 °C).
5. **It doesn't meet the target on its own at the default clock** (+2.9 ms over legacy). The key exchange halves (X25519 ‖ ML-KEM) are still sequential inside OpenSSL, and part 1 showed another ~2.1 ms is available there. Combined with the other approaches (min 750 MHz, bitmanip) the target should be met with margin; that isn't measured yet.

Limits and notes

- **Own server only.** The hackathon server can't do hybrid authentication, so this can't be measured against it.
- **Not a standard.** `0xFE20` is a private-use extension number; a production version would use the IETF composite ML-DSA certificates or a registered extension. The construction (signature over the hello messages) should be reviewed before real use.
- In `par` mode the client completes the TLS handshake (sends its Finished) before the ML-DSA result is known; it sends no application data until both signatures have verified. The `v2_seq` mode aborts inside the handshake instead.
- HelloRetryRequest isn't handled (it doesn't occur with this server; the signature would then need to cover both ClientHellos).
- Session resumption skips certificates, so v2 adds nothing there (and costs nothing).

Source: multicore_e2e_REPORT.md